

BÁO CÁO ĐỀ TÀI

NHẬN DẠNG MÃ MORSE TỪ ÂM THANH BẰNG DEEP LEARNING

MỤC LỤC

1. Tổng quan đề tài
2. Phạm vi và yêu cầu bài toán
3. Cơ sở lý thuyết về mã Morse âm thanh
4. Chuẩn bị dữ liệu synthetic
5. Tiền xử lý âm thanh và trích xuất đặc trưng
6. Định dạng dữ liệu đầu vào của mô hình
7. Kiến trúc mô hình Conv1D + Bi-LSTM + CTC
8. Quy trình huấn luyện mô hình
9. Hệ thống đánh giá và metrics
10. Quy trình inference và lưu kết quả
11. Kết quả thực nghiệm
12. Cấu trúc thư mục và tổ chức trên GCS
13. Hướng phát triển với dữ liệu real
14. Khó khăn, rủi ro và giải pháp
15. Kết luận

1. Tổng quan đề tài

Đề tài xây dựng hệ thống nhận dạng mã Morse từ tín hiệu âm thanh bằng mô hình học sâu. Hệ thống nhận đầu vào là file âm thanh chứa tiếng bíp Morse, chuyển đổi tín hiệu thành đặc trưng dạng spectrogram, sau đó sử dụng mô hình học chuỗi để dự đoán trực tiếp nội dung văn bản tương ứng.

Trong phạm vi hiện tại, toàn bộ quá trình huấn luyện và đánh giá được thực hiện trên dữ liệu âm thanh Morse tổng hợp. Dữ liệu tổng hợp giúp kiểm soát chính xác transcript, tốc độ WPM, tần số tone, mức nhiễu và độ dài audio, nhờ đó phù hợp cho việc xây dựng pipeline huấn luyện ổn định trước khi mở rộng sang dữ liệu thực tế.

Thành phần	Mô tả
Bài toán	Nhận dạng văn bản từ audio Morse
Loại dữ liệu	Âm thanh WAV mono, sample rate 16 kHz
Nguồn huấn luyện	Dữ liệu synthetic Morse audio
Phương pháp	End-to-end sequence recognition với CTC Loss
Kết quả mong muốn	Dự đoán đúng transcript từ file audio đầu vào

2. Phạm vi và yêu cầu bài toán

2.1. Yêu cầu chức năng

- Sinh được tập audio test synthetic và lưu vào Google Cloud Storage theo cấu trúc thống nhất.
- Tải một file audio bất kỳ, trích xuất đặc trưng spectrogram và đưa vào mô hình.
- Mô hình dự đoán chuỗi văn bản từ audio bằng greedy CTC decoding.
- So sánh kết quả dự đoán với transcript tham chiếu nếu có.
- Lưu audio, spectrogram và kết quả dự đoán vào thư mục test/audio/<tên audio>/ trên GCS.

2.2. Yêu cầu phi chức năng

- Dữ liệu và checkpoint được lưu trữ trên Google Cloud Storage.
- Mã nguồn dễ chạy lại, ít phụ thuộc vào thao tác thủ công.

3. Cơ sở lý thuyết về mã Morse âm thanh

Mã Morse biểu diễn ký tự bằng các tín hiệu ngắn và dài, thường gọi là dot và dash. Trong âm thanh, mỗi dot hoặc dash được thể hiện bằng một tone liên tục ở tần số cố định trong một khoảng thời gian nhất định. Khoảng im lặng giữa các tone giúp phân tách thành phần trong ký tự, giữa các ký tự và giữa các từ.

Thành phần Morse	Ký hiệu	Thời lượng tương đối
Dot	.	1 đơn vị thời gian
Dash	-	3 đơn vị thời gian
Gap trong cùng ký tự	Im lặng	1 đơn vị thời gian
Gap giữa hai ký tự	Im lặng	3 đơn vị thời gian
Gap giữa hai từ	Im lặng	7 đơn vị thời gian

Tốc độ Morse thường đo bằng WPM (Words Per Minute). Với cách sinh dữ liệu trong đề tài, thời lượng dot được tính theo công thức: $\text{dot} = 1.2 / \text{WPM}$. Khi WPM tăng, dot ngắn hơn, audio ngắn hơn và mật độ tín hiệu theo thời gian cao hơn.

```
dot_duration = 1.2 / WPM
dash_duration = 3 * dot_duration
word_gap = 7 * dot_duration
```

4. Chuẩn bị dữ liệu synthetic

Dữ liệu huấn luyện chính là tập synthetic_realistic_v1 gồm 50.000 file audio Morse tổng hợp. Mỗi file có transcript chính xác vì audio được sinh trực tiếp từ văn bản. Cách tiếp cận này loại bỏ vấn đề sai lệch nhãn theo thời gian và phù hợp với mục tiêu xây dựng mô hình nhận dạng trên dữ liệu tổng hợp.

Thuộc tính	Giá trị sử dụng
Số mẫu	50.000 audio clips
Định dạng audio	WAV, mono
Sample rate	16.000 Hz
Độ dài trung vị	Khoảng 12,33 giây
Độ dài nhỏ nhất	Khoảng 6,58 giây
Độ dài lớn nhất	Khoảng 15,50 giây
Tần số tone chính	Quanh 800 Hz
Label	Transcript văn bản, gồm A-Z, 0-9 và dấu cách

WPM	Số lượng mẫu
10	1.218
13	2.029
15	2.969
18	6.469
20	6.635
25	8.097
30	8.127
35	8.371
40	6.085

4.1. Quy trình sinh dữ liệu

1. Chọn một đoạn văn bản đầu vào và chuẩn hóa về chữ in hoa.
2. Loại bỏ ký tự không thuộc tập A-Z, 0-9 và dấu cách.

3. Chuyển từng ký tự thành chuỗi dot/dash theo bảng mã Morse quốc tế.
4. Sinh tone âm thanh cho dot và dash theo WPM tương ứng.
5. Thêm khoảng im lặng giữa thành phần, giữa ký tự và giữa từ.
6. Thêm nhiễu trắng ở mức SNR được cấu hình để tăng độ đa dạng.
7. Lưu file WAV và metadata transcript tương ứng.

4.2. Lý do dùng dữ liệu synthetic

- Label chính xác tuyệt đối vì transcript là nguồn sinh audio.
- Dễ kiểm soát độ dài, tốc độ WPM, mức nhiễu và tần số tone.
- Có thể tạo số lượng lớn mà không cần gán nhãn thủ công.
- Phù hợp để đánh giá mô hình trong điều kiện có ground truth rõ ràng.
- Giúp tập trung vào bài toán học chuỗi âm thanh thay vì xử lý lỗi alignment của dữ liệu thực.

5. Tiền xử lý âm thanh và trích xuất đặc trưng

Mỗi file audio được chuyển thành đặc trưng dạng narrow spectrogram. Thay vì dùng toàn bộ phổ tần rộng như bài toán nhận dạng tiếng nói, hệ thống chỉ lấy 21 frequency bins quanh tần số tone Morse 800 Hz. Lý do là tín hiệu Morse về bản chất là một tone đơn bật/tắt theo thời gian, nên thông tin quan trọng nằm ở sự xuất hiện và biến mất của năng lượng quanh tần số tone.

Tham số	Giá trị	Ý nghĩa
target_sr	16000	Chuẩn hóa sample rate của audio
n_fft	512	Kích thước cửa sổ FFT
hop_length	128	Bước trượt giữa hai frame liên tiếp
target_freq	800 Hz	Tần số trung tâm cần quan sát
n_bins	21	Số frequency bins lấy quanh 800 Hz
output shape	(T, 21)	T là số frame theo thời gian

5.1. CÁC BƯỚC XỬ LÝ ÂM THANH THÀNH SPECTROGRAM

Trước khi đưa file âm thanh vào mô hình học sâu, tín hiệu Morse cần được chuyển từ dạng waveform thô sang dạng đặc trưng số phù hợp hơn cho mô hình. Trong đề tài này, đặc trưng được sử dụng là narrow-band spectrogram, tức là spectrogram dải hẹp quanh tần số tone Morse chính. Cụ thể, hệ thống lấy 21 frequency bins xung quanh tần số 800 Hz. Đây là tần số trung tâm được sử dụng khi sinh dữ liệu synthetic Morse.

Việc dùng spectrogram giúp mô hình quan sát rõ hơn sự thay đổi năng lượng của tín hiệu Morse theo thời gian. Trong waveform thô, thông tin dot, dash và khoảng lặng bị trộn trong chuỗi dao động biên độ rất dài. Trong spectrogram, các đoạn có tone Morse sẽ thể hiện bằng vùng năng lượng mạnh quanh 800 Hz, còn các khoảng lặng sẽ có năng lượng thấp hơn. Do đó, spectrogram phù hợp hơn để mô hình học quy luật bật/tắt của tín hiệu Morse.

Quy trình xử lý một file audio gồm các bước sau:

Bước 1. Đọc file WAV bằng thư viện soundfile

Mỗi file âm thanh đầu vào được đọc bằng thư viện soundfile. Sau khi đọc, hệ thống thu được hai thành phần chính:

- y : mảng waveform biểu diễn biên độ âm thanh theo thời gian.
- sr : sample rate, tức số mẫu âm thanh trên một giây.

Ví dụ, nếu $sr = 16000$ Hz, nghĩa là mỗi giây âm thanh được biểu diễn bằng 16.000 điểm mẫu. Ở bước này, dữ liệu vẫn còn là waveform thô. Waveform phù hợp để lưu trữ âm thanh, nhưng chưa phải dạng đầu vào tối ưu cho mô hình nhận dạng Morse. Vì vậy, waveform sẽ tiếp tục được xử lý để chuyển sang biểu diễn spectrogram.

Bước 2. Chuyển audio nhiều kênh về mono

Một số file âm thanh có thể có nhiều kênh, ví dụ stereo gồm kênh trái và kênh phải. Tuy nhiên, trong bài toán nhận dạng mã Morse, thông tin quan trọng không nằm ở vị trí âm thanh bên trái hay bên phải, mà nằm ở chuỗi tone Morse xuất hiện theo thời gian.

Do đó, nếu audio có nhiều kênh, hệ thống chuyển về mono bằng cách lấy trung bình các kênh. Sau bước này, mỗi file âm thanh chỉ còn một chuỗi waveform duy nhất.

Việc chuyển về mono có các lợi ích sau:

- Giảm kích thước dữ liệu cần xử lý.
- Đưa tất cả file audio về cùng một định dạng thống nhất.
- Loại bỏ thông tin kênh không cần thiết đối với bài toán nhận dạng Morse.
- Giúp mô hình chỉ tập trung vào tín hiệu tone theo thời gian.

Bước 3. Chuẩn hóa sample rate về 16 kHz

Các file âm thanh có thể có sample rate khác nhau, ví dụ 8 kHz, 16 kHz, 22.05 kHz hoặc 44.1 kHz. Nếu giữ nguyên các sample rate khác nhau, cùng một đoạn âm thanh Morse có thể tạo ra số lượng frame khác nhau khi tính spectrogram. Điều này làm dữ liệu thiếu nhất quán và gây khó khăn cho quá trình huấn luyện.

Vì vậy, toàn bộ audio được chuẩn hóa về sample rate 16 kHz. Nếu file gốc đã có sample rate 16 kHz thì giữ nguyên. Nếu file có sample rate khác, hệ thống thực hiện resample về 16 kHz bằng thư viện librosa.

Việc chọn 16 kHz là phù hợp vì tín hiệu Morse trong đề tài chỉ nằm quanh một tone đơn, cụ thể là 800 Hz. Do đó, không cần dùng sample rate quá cao như 44.1 kHz. Sample rate 16 kHz vẫn đủ để biểu diễn tốt tín hiệu Morse, đồng thời giúp giảm dung lượng dữ liệu và tăng tốc độ xử lý.

Bước 4. Tính STFT để thu được ma trận phổ biên độ theo thời gian

Sau khi waveform đã được chuẩn hóa, hệ thống áp dụng phép biến đổi Fourier thời gian ngắn, gọi là STFT. STFT chia tín hiệu âm thanh thành nhiều cửa sổ thời gian nhỏ. Với mỗi cửa sổ, hệ thống tính phổ tần số để biết tại thời điểm đó tín hiệu có năng lượng mạnh ở tần số nào.

Trong đề tài này, STFT sử dụng các tham số:

- $n_fft = 512$
- $hop_length = 128$

Tham số $n_fft = 512$ nghĩa là mỗi lần phân tích, hệ thống xét một cửa sổ gồm 512 mẫu âm thanh. Tham số $hop_length = 128$ nghĩa là hai cửa sổ liên tiếp cách nhau 128 mẫu.

Với sample rate 16 kHz, khoảng cách thời gian giữa hai frame liên tiếp là:

$$128 / 16000 = 0.008 \text{ giây}$$

Tức là cứ khoảng 8 ms, hệ thống tạo ra một frame đặc trưng mới. Mức phân giải thời gian này đủ nhỏ để mô hình quan sát được sự khác biệt giữa dot, dash và các khoảng nghỉ trong mã Morse.

Kết quả của STFT là một ma trận phổ phức. Trong bài toán này, hệ thống chỉ lấy phần biên độ của phổ:

Bước 5. Xác định frequency bin gần 800 Hz nhất

Trong dữ liệu synthetic của đề tài, tone Morse được sinh quanh tần số 800 Hz. Vì vậy, sau khi tính STFT, hệ thống cần xác định frequency bin nào gần với 800 Hz nhất.

Đầu tiên, hệ thống tạo danh sách các tần số tương ứng với từng frequency bin trong STFT. Sau đó, tìm bin có khoảng cách nhỏ nhất tới 800 Hz:

Bin này được xem là trung tâm của vùng tần số chứa tín hiệu Morse chính.

Việc xác định bin trung tâm giúp hệ thống tập trung vào đúng vùng có tone Morse. Nếu sử dụng toàn bộ phổ tần số, mô hình sẽ phải xử lý thêm rất nhiều vùng không chứa thông tin hữu ích. Điều này làm tăng kích thước đầu vào, tăng thời gian huấn luyện và có thể khiến mô hình học phải nhiễu không cần thiết.

Bước 6. Lấy 21 frequency bins xung quanh bin trung tâm

Sau khi xác định được bin tần số gần 800 Hz nhất, hệ thống lấy 21 frequency bins xung quanh bin này. Cụ thể, lấy 10 bins phía dưới, 1 bin trung tâm và 10 bins phía trên.

Như vậy, mỗi frame thời gian được biểu diễn bằng 21 giá trị phổ. Đầu ra sau bước này có dạng:

S_narrow có kích thước: $21 \times T$

Trong đó:

21 là số frequency bins quanh tone 800 Hz.

T là số frame theo thời gian.

Sau đó, ma trận sẽ được chuyển vị để đưa vào mô hình dưới dạng: $(T, 21)$. Trong đó mỗi dòng là một frame thời gian, và mỗi frame là một vector 21 chiều.

Việc chỉ lấy 21 bins quanh tần số trung tâm có nhiều ưu điểm:

- Giảm kích thước dữ liệu đầu vào.
- Loại bỏ phần lớn vùng tần số không liên quan.
- Giúp mô hình tập trung vào sự xuất hiện và biến mất của tone Morse.
- Tăng tốc độ huấn luyện và suy luận.
- Giảm nguy cơ overfitting vào nhiễu ngoài dải tần quan tâm.

Khác với nhận dạng giọng nói, tín hiệu Morse không có cấu trúc phổ phức tạp gồm nhiều formant hoặc harmonic. Morse trong bài toán này chủ yếu là một tone đơn được bật và tắt theo thời gian. Vì vậy, dải phổ hẹp quanh 800 Hz là đủ để biểu diễn thông tin cần thiết.

Bước 7. Áp dụng log1p để giảm chênh lệch biên độ

Sau khi lấy vùng phổ hẹp, các giá trị biên độ trong spectrogram có thể chênh lệch rất lớn. Các frame có tone Morse mạnh sẽ có giá trị lớn, trong khi các frame là khoảng lặng hoặc nhiễu nhẹ sẽ có giá trị nhỏ hơn nhiều.

Nếu đưa trực tiếp biên độ thô vào mô hình, các giá trị lớn có thể chi phối quá trình học. Do đó, hệ thống áp dụng phép biến đổi logarit:

```
S_log = log(1 + S_narrow)
```

Trong code, phép biến đổi này được thực hiện bằng hàm log1p.

Tác dụng của log1p là nén các giá trị biên độ lớn và làm cho phân phối dữ liệu ổn định hơn. Sau bước này, sự khác biệt giữa các frame có năng lượng rất mạnh và các frame có năng lượng trung bình sẽ bớt cực đoan hơn. Điều này giúp mô hình học ổn định và tránh phụ thuộc quá nhiều vào âm lượng tuyệt đối của tín hiệu.

Bước 8. Chuẩn hóa từng frequency bin về zero-mean và unit-variance

Sau khi áp dụng log1p, dữ liệu vẫn có thể có phân phối khác nhau giữa các frequency bins. Các bin gần tần số trung tâm thường có năng lượng cao hơn, trong khi các bin xa hơn có năng lượng thấp hơn. Nếu không chuẩn hóa, mô hình có thể bị lệch về các bin có biên độ lớn.

Vì vậy, hệ thống chuẩn hóa từng frequency bin theo thời gian bằng công thức:

```
S_norm = (S_log - mean) / std
```

Trong đó:

mean là giá trị trung bình của một frequency bin theo toàn bộ thời gian.

std là độ lệch chuẩn của frequency bin đó theo toàn bộ thời gian.

Sau chuẩn hóa, mỗi frequency bin có phân phối xấp xỉ mean ≈ 0 và std ≈ 1 .

Bước chuẩn hóa này giúp dữ liệu đầu vào ổn định hơn, quá trình train hội tụ nhanh hơn, giảm ảnh hưởng của khác biệt âm lượng giữa các file audio, giúp các frequency bins đóng góp cân bằng hơn cho mô hình và ổn định gradient trong quá trình huấn luyện.

Bước 9. Clip giá trị về khoảng [-3, 3]

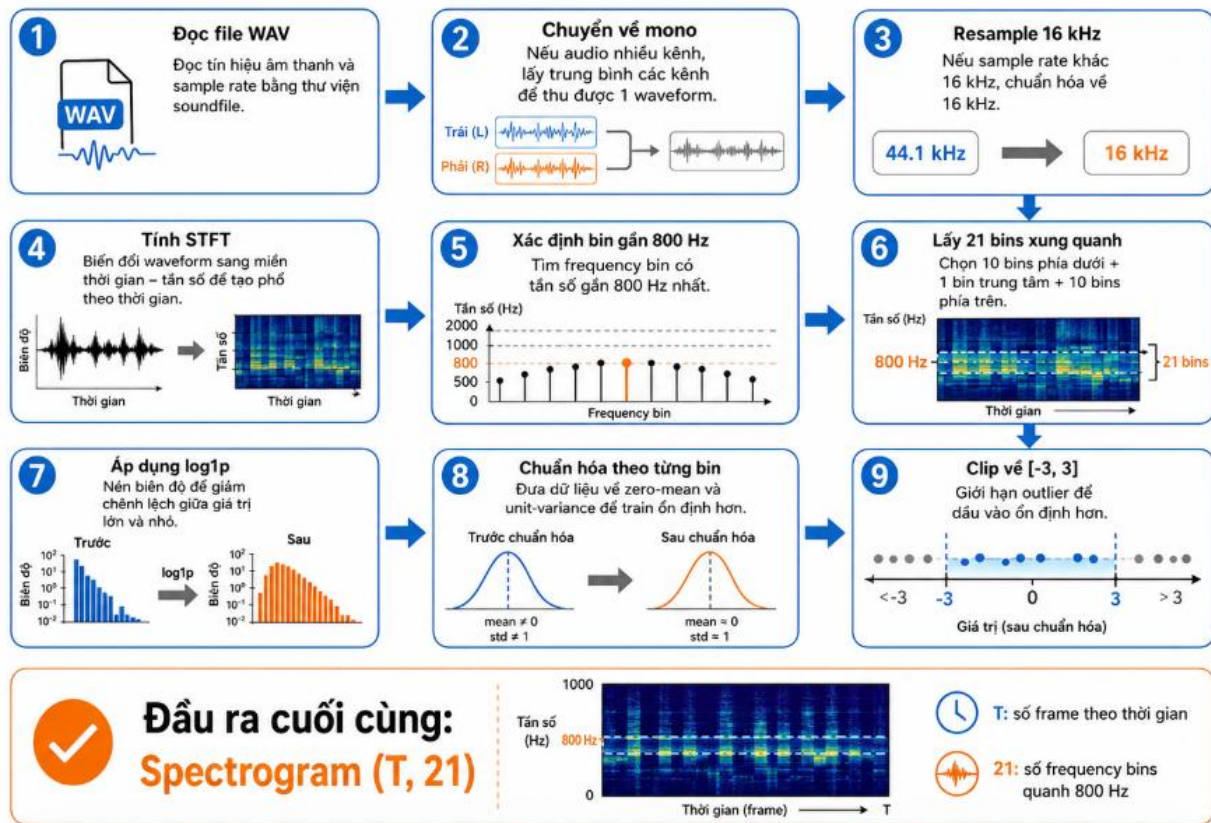
Sau khi chuẩn hóa, một số giá trị vẫn có thể rất lớn hoặc rất nhỏ do nhiễu, clipping hoặc các peak bất thường trong audio. Những giá trị ngoại lệ này có thể làm quá trình huấn luyện kém ổn định.

Vì vậy, hệ thống giới hạn giá trị spectrogram trong khoảng [-3, 3]. Cụ thể: nếu giá trị lớn hơn 3 thì đưa về 3; nếu giá trị nhỏ hơn -3 thì đưa về -3.

Việc clip dữ liệu giúp loại bỏ ảnh hưởng của outlier, ổn định đầu vào cho mô hình và giảm nguy cơ xuất hiện gradient bất thường trong quá trình train. Khoảng [-3, 3] là hợp lý vì sau chuẩn hóa z-score, phần lớn dữ liệu thường nằm trong khoảng này. Các giá trị nằm ngoài khoảng này thường là ngoại lệ.

CÁC BƯỚC XỬ LÝ ÂM THANH THÀNH SPECTROGRAM

Quy trình xử lý audio cho bài toán nhận dạng Morse



Kết quả sau xử lý

Sau toàn bộ các bước trên, mỗi file audio Morse được chuyển thành một ma trận đặc trưng có dạng:

$(T, 21)$

Trong đó:

- T là số frame thời gian, phụ thuộc vào độ dài audio.
- 21 là số frequency bins quanh tần số 800 Hz.

Ví dụ, với audio dài khoảng 12 giây, sample rate 16 kHz và hop_length = 128, số frame xấp xỉ là:

$$T \approx 12 \times 16000 / 128 = 1500 \text{ frame}$$

Do đó, một file audio 12 giây sẽ tạo ra spectrogram có kích thước xấp xỉ:

$(1500, 21)$

Đây là đầu vào trực tiếp của mô hình CTC.

Ý nghĩa của Spectrogram đối với mô hình

Spectrogram sau xử lý được xem như một chuỗi theo thời gian:

$x = [\text{frame}_1, \text{frame}_2, \dots, \text{frame}_T]$

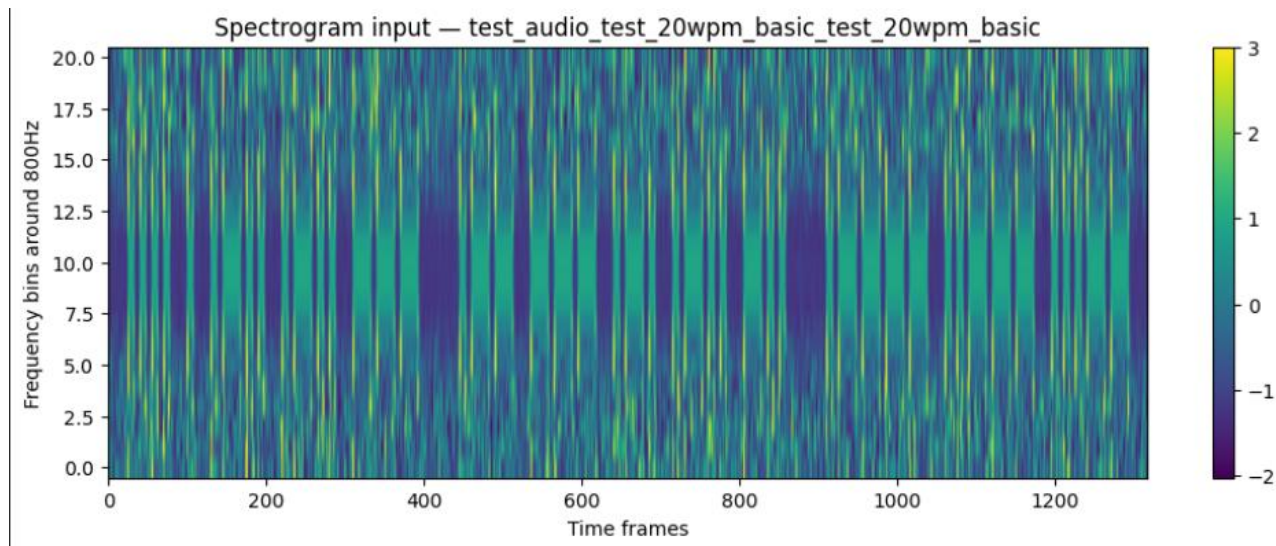
Mỗi frame là một vector 21 chiều:

$\text{frame}_t \in \mathbb{R}^{21}$

Mô hình sẽ học ánh xạ từ chuỗi vector này sang chuỗi ký tự văn bản:

Spectrogram (T, 21) → Text

Do độ dài audio và độ dài transcript không giống nhau, mô hình sử dụng CTC Loss để học mà không cần biết chính xác mỗi ký tự xuất hiện tại thời điểm nào. Điều này rất phù hợp với bài toán Morse, vì ta chỉ cần transcript cuối cùng mà không cần gán nhãn timestamp cho từng dot, dash hoặc từng ký tự.



6. Định dạng dữ liệu đầu vào của mô hình

Mô hình nhận đầu vào là tensor có kích thước (B, T, 21), trong đó B là batch size, T là số frame thời gian và 21 là số frequency bins được lấy quanh tần số tone Morse. Định dạng này giữ được thông tin biến thiên theo thời gian, đồng thời giảm số chiều đặc trưng để mô hình nhỏ và train nhanh.

Chiều	Ý nghĩa
B	Số mẫu trong một batch
T	Số frame theo thời gian, thay đổi theo độ dài audio
21	Số bins tần số quanh 800 Hz

6.1. Định dạng label

Label của mô hình là transcript văn bản đã chuẩn hóa, không phải chuỗi dot/dash. Vocabulary gồm 26 chữ cái Latin, 10 chữ số, dấu cách và một token blank cho CTC.

Nhóm token	Nội dung	Số lượng
Chữ cái	A-Z	26
Chữ số	0-9	10
Dấu cách	Space	1
CTC blank	<blank>	1
Tổng	Vocabulary output	38

6.2. Vì sao dự đoán trực tiếp văn bản

- Giảm một bước xử lý so với cách dự đoán dot/dash rồi mới decode sang text.
- CTC cho phép học alignment ngầm giữa frame âm thanh và chuỗi ký tự.
- Label transcript dễ tạo, dễ đánh giá bằng CER và WER.
- Phù hợp với bài toán end-to-end audio-to-text.

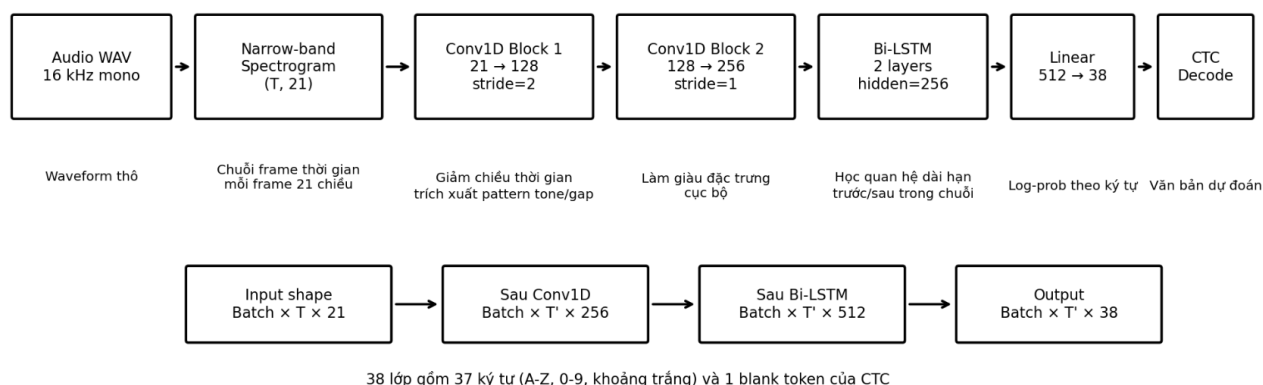
7. Kiến trúc mô hình Conv1D + Bi-LSTM + CTC

Mô hình được sử dụng trong đề tài là mô hình nhận dạng chuỗi âm thanh dựa trên kiến trúc kết hợp giữa **Conv1D**, **Bi-LSTM** và **CTC**. Mục tiêu của mô hình là nhận đầu vào là spectrogram của tín hiệu Morse và dự đoán trực tiếp chuỗi ký tự văn bản tương ứng.

Pipeline tổng quát của mô hình như sau:

Audio WAV $\rightarrow$ *Spectrogram (T, 21)* $\rightarrow$ *Conv1D* $\rightarrow$ *Bi-LSTM* $\rightarrow$ *Linear* $\rightarrow$ *CTC Decode* $\rightarrow$ *Text*

Kiến trúc mô hình Conv1D + Bi-LSTM + CTC



Trong đó:

- T là số frame thời gian của audio.
- 21 là số frequency bins quanh tần số 800 Hz.
- Text là chuỗi ký tự đầu ra gồm chữ cái, chữ số và khoảng trắng.

Mô hình không cần biết chính xác mỗi ký tự xuất hiện tại thời điểm nào trong audio. Thay vào đó, mô hình sử dụng CTC Loss để học ánh xạ từ chuỗi frame âm thanh sang chuỗi văn bản cuối cùng.

7.1. Đầu vào của mô hình

Đầu vào của mô hình là spectrogram đã được xử lý từ file audio Morse. Sau bước tiền xử lý, mỗi audio được biểu diễn dưới dạng ma trận: (T, 21)

Trong đó:

- T là số frame theo thời gian. Giá trị này phụ thuộc vào độ dài audio.
- 21 là số frequency bins được lấy quanh tone Morse 800 Hz.
- Ví dụ, với audio dài khoảng 12 giây, sample rate 16 kHz và hop_length = 128, số frame xấp xỉ là:
 $T \approx 12 \times 16000 / 128 = 1500$ frame

Như vậy, đầu vào có thể có kích thước khoảng: (1500, 21)

Khi đưa vào mô hình theo batch, tensor có dạng: (batch_size, T, 21)

Đây là dạng dữ liệu phù hợp cho mô hình sequence learning, vì mỗi frame thời gian được xem như một vector đặc trưng 21 chiều. Mô hình sẽ xử lý chuỗi các vector này để suy ra chuỗi ký tự Morse tương ứng.

7.2. Conv1D

Lớp Conv1D được sử dụng để trích xuất các mẫu cục bộ trong chuỗi spectrogram. Trong tín hiệu Morse, các thông tin quan trọng như dot, dash và khoảng lặng đều thể hiện qua các đoạn năng lượng ngắn theo thời gian. Do đó, Conv1D rất phù hợp để học các đặc trưng cục bộ này.

Conv1D hoạt động dọc theo trục thời gian. Mỗi kernel của Conv1D quét qua các frame liên tiếp để phát hiện các pattern ngắn, ví dụ:

- sự xuất hiện của tone,
- sự biến mất của tone,
- đoạn tone ngắn giống dot,
- đoạn tone dài giống dash,
- khoảng lặng giữa các ký tự hoặc giữa các từ.

Trong mô hình, khối Conv1D gồm hai lớp chính:

Lớp Conv1D thứ nhất:

- Input channels: 21
- Output channels: 128
- Kernel size: 5
- Stride: 2
- Padding: 2

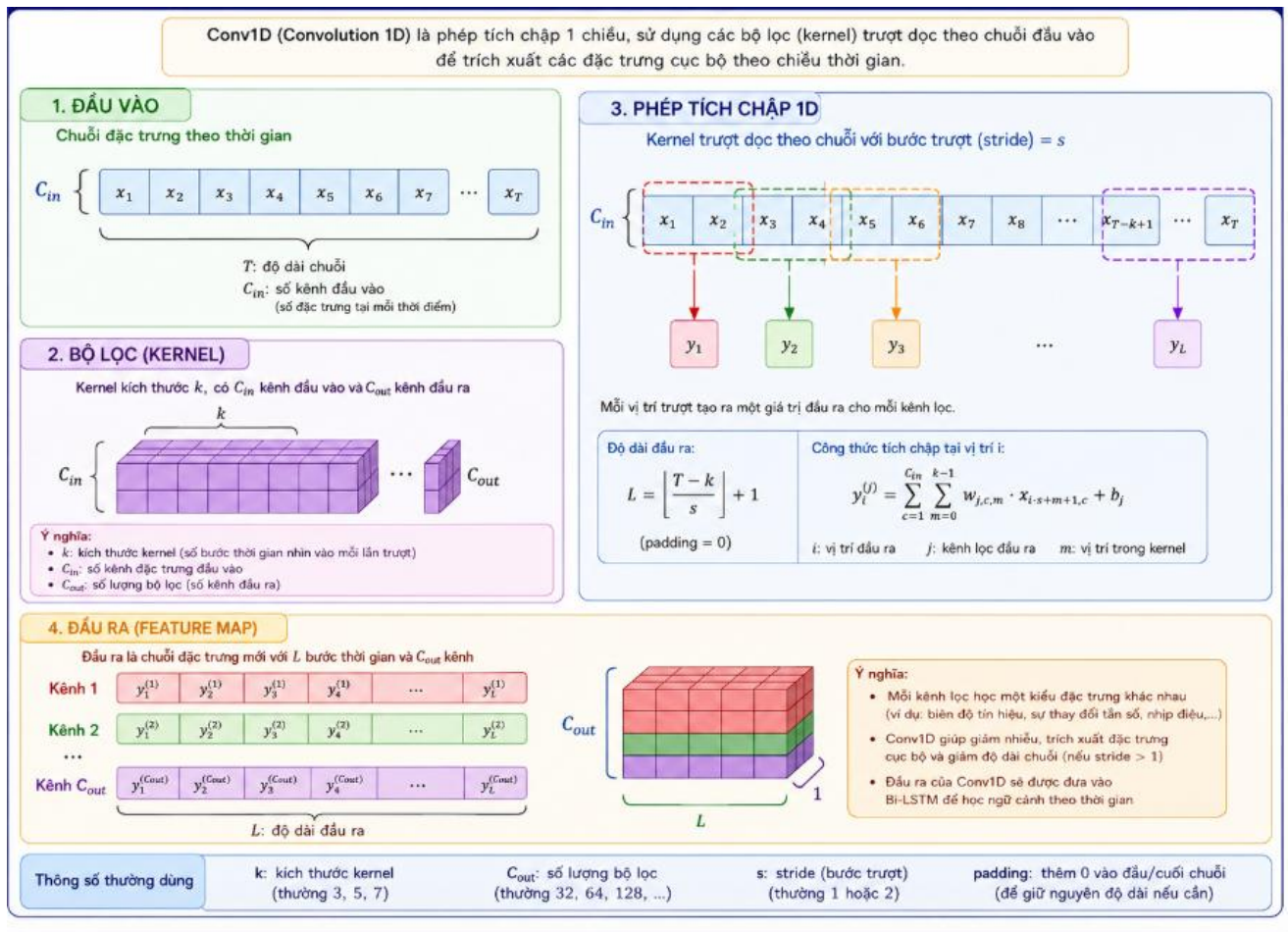
Lớp này vừa trích xuất đặc trưng cục bộ, vừa giảm chiều dài chuỗi thời gian. Việc sử dụng stride = 2 giúp giảm số frame cần xử lý ở các tầng sau, làm mô hình train nhanh hơn và tiết kiệm bộ nhớ hơn.

Lớp Conv1D thứ hai:

- Input channels: 128
- Output channels: 256
- Kernel size: 5
- Stride: 1
- Padding: 2

Lớp này tiếp tục làm giàu biểu diễn đặc trưng, nhưng không giảm thêm chiều dài thời gian. Sau hai lớp Conv1D, mỗi frame thời gian được biểu diễn bằng vector đặc trưng 256 chiều.

Mỗi lớp Conv1D được kết hợp với BatchNorm1D, ReLU và Dropout. BatchNorm giúp ổn định phân phối đặc trưng, ReLU tạo tính phi tuyến, còn Dropout giúp giảm overfitting.



Vai trò của BatchNorm, ReLU và Dropout

Sau mỗi lớp Conv1D, mô hình sử dụng ba thành phần phụ trợ:

- BatchNorm1D**
 BatchNorm1D chuẩn hóa đầu ra của lớp Conv1D theo batch. Điều này giúp mô hình huấn luyện ổn định hơn, giảm hiện tượng gradient dao động mạnh và giúp quá trình hội tụ nhanh hơn.
- ReLU**
 ReLU là hàm kích hoạt phi tuyến. Nếu chỉ dùng các phép biến đổi tuyến tính, mô hình sẽ khó học được quan hệ phức tạp trong tín hiệu Morse. ReLU giúp mô hình học được các pattern phi tuyến giữa năng lượng phổ và cấu trúc dot/dash.
- Dropout**
 Dropout ngẫu nhiên loại bỏ một phần đặc trưng trong quá trình train. Điều này giúp mô hình không phụ thuộc quá mức vào một vài đặc trưng cụ thể, từ đó giảm overfitting và tăng khả năng tổng quát hóa trên các audio synthetic khác nhau.

7.4. Bi-LSTM để học quan hệ theo thời gian

LSTM, (Long Short-Term Memory), là một biến thể đặc biệt của mạng nơ-ron hồi tiếp **RNN**. LSTM được thiết kế để xử lý dữ liệu dạng chuỗi, tức là dữ liệu có thứ tự theo thời gian hoặc theo vị trí. Trong bài toán nhận dạng mã Morse từ audio, đầu vào của mô hình là một chuỗi các frame spectrogram theo thời gian, vì vậy **LSTM** là một kiến trúc phù hợp để học mối quan hệ giữa các frame liên tiếp.

Khác với mạng neural network thông thường, **LSTM** không xử lý từng frame một cách độc lập. Thay vào đó, tại mỗi thời điểm, **LSTM** vừa nhận frame hiện tại, vừa sử dụng thông tin đã ghi nhớ từ các thời điểm trước đó. Nhờ vậy, **LSTM** có thể học được các đặc trưng có tính thời gian như độ dài của tone, khoảng cách giữa các tone, khoảng cách giữa các ký tự và khoảng cách giữa các từ.

Trong tín hiệu Morse, một ký tự không thể được nhận biết chính xác chỉ từ một frame đơn lẻ. Ví dụ, để phân biệt dot và dash, mô hình cần biết tone kéo dài trong bao lâu. Dot là tone ngắn, còn dash là tone dài. Tương tự, để phân biệt khoảng cách giữa hai ký tự và khoảng cách giữa hai từ, mô hình cần quan sát một đoạn thời gian liên tục chứ không thể chỉ nhìn từng thời điểm riêng lẻ. Đây chính là lý do **LSTM** được sử dụng trong bài toán này.

Cấu trúc bên trong của LSTM

Một LSTM cell tại thời điểm t nhận ba thông tin chính:

- x_t : đặc trưng đầu vào tại thời điểm t , trong bài toán này là một frame spectrogram.
- h_{t-1} : trạng thái ẩn từ thời điểm trước đó.
- c_{t-1} : cell state, hay còn gọi là bộ nhớ dài hạn từ thời điểm trước đó.

Sau khi xử lý, LSTM tạo ra:

- h_t : trạng thái ẩn mới tại thời điểm t .
- c_t : cell state mới, chứa thông tin được cập nhật sau khi đọc frame hiện tại.

Điểm quan trọng nhất của LSTM là nó có cơ chế bộ nhớ. Bộ nhớ này cho phép mô hình quyết định thông tin nào nên được giữ lại, thông tin nào nên bị quên đi và thông tin mới nào nên được thêm vào.

Một LSTM cell gồm các thành phần chính sau:

- **Forget Gate**

Forget gate quyết định phần thông tin cũ nào trong bộ nhớ c_{t-1} cần được giữ lại hoặc loại bỏ. Nếu một thông tin không còn quan trọng cho việc dự đoán các frame tiếp theo, LSTM có thể giảm ảnh hưởng của thông tin đó.

Trong bài toán Morse, forget gate giúp mô hình loại bỏ các tín hiệu không còn cần thiết, ví dụ như một đoạn tone đã kết thúc hoặc một đoạn nhiễu ngắn không liên quan đến ký tự đang được nhận dạng.

- **Input Gate**

Input gate quyết định thông tin mới nào từ frame hiện tại x_t sẽ được đưa vào bộ nhớ. Không phải mọi thông tin từ frame hiện tại đều quan trọng. Input gate giúp mô hình chọn lọc thông tin cần ghi nhớ.

Trong bài toán Morse, input gate có thể học cách chú ý đến sự xuất hiện của tone, sự thay đổi từ khoảng lặng sang tone, hoặc sự thay đổi từ tone sang khoảng lặng.

- **Candidate State**

Candidate state là phần thông tin mới được tạo ra từ đầu vào hiện tại và trạng thái ẩn trước đó. Đây là nội dung có khả năng được thêm vào bộ nhớ nếu input gate cho phép.

Trong nhận dạng Morse, candidate state có thể chứa thông tin như “tone đang tiếp tục”, “tone vừa bắt đầu”, “tone vừa kết thúc” hoặc “đang có khoảng lặng”.

- **Cell State**

Cell state là bộ nhớ dài hạn của LSTM. Đây là thành phần giúp LSTM ghi nhớ thông tin qua nhiều bước thời gian. Cell state có thể mang thông tin từ các frame trước đến các frame sau, nhờ đó mô hình không bị mất ngữ cảnh khi xử lý chuỗi dài.

Trong bài toán Morse, cell state rất quan trọng vì mô hình cần ghi nhớ độ dài của tone và độ dài của khoảng lặng. Một tone chỉ được xác định là dot hay dash sau khi mô hình quan sát đủ thời gian kéo dài của nó.

- **Output Gate**

Output gate quyết định phần nào của cell state sẽ được đưa ra ngoài dưới dạng trạng thái ẩn h_t . Trạng thái ẩn này sẽ được truyền sang thời điểm tiếp theo và cũng được dùng để dự đoán đầu ra tại thời điểm hiện tại.

Trong bài toán Morse, output gate giúp mô hình tạo ra biểu diễn phù hợp cho từng frame, từ đó hỗ trợ lớp phía sau dự đoán ký tự hoặc blank token trong CTC.

Cách LSTM hoạt động theo thời gian

LSTM xử lý chuỗi đầu vào theo từng frame thời gian. Với chuỗi spectrogram:

$$x_1, x_2, x_3, \dots, x_T$$

LSTM sẽ đọc lần lượt từng frame. Tại mỗi thời điểm t , LSTM thực hiện các bước:

- Nhận frame hiện tại x_t .
- Nhận trạng thái ẩn trước đó h_{t-1} .
- Nhận bộ nhớ trước đó c_{t-1} .
- Dùng forget gate để quyết định quên phần nào của bộ nhớ cũ.
- Dùng input gate để quyết định ghi thêm thông tin mới nào.
- Cập nhật cell state thành c_t .
- Dùng output gate để tạo trạng thái ẩn mới h_t .
- Truyền h_t và c_t sang thời điểm tiếp theo.

Nhờ quá trình này, LSTM có khả năng học được mối quan hệ phụ thuộc theo thời gian. Điều này đặc biệt quan trọng với tín hiệu Morse, vì ý nghĩa của tín hiệu phụ thuộc mạnh vào độ dài và thứ tự của các đoạn tone và khoảng lặng.

Ví dụ, nếu mô hình nhìn thấy một tone xuất hiện trong một thời gian ngắn, nó có thể học rằng đây có khả năng là dot. Nếu tone kéo dài lâu hơn, mô hình có thể học rằng đây có khả năng là dash. Nếu sau tone là một khoảng lặng ngắn, đó có thể là khoảng cách trong cùng một ký tự. Nếu khoảng lặng dài hơn, đó có thể là khoảng cách giữa hai ký tự hoặc hai từ.

Bi-LSTM là gì?

Bi-LSTM, viết đầy đủ là Bidirectional Long Short-Term Memory, là phiên bản hai chiều của **LSTM**. Thay vì chỉ đọc chuỗi theo một hướng từ đầu đến cuối, **Bi-LSTM** xử lý chuỗi theo cả hai hướng:

- Hướng xuôi: từ trái sang phải, tức từ thời điểm đầu audio đến thời điểm cuối audio.
- Hướng ngược: từ phải sang trái, tức từ thời điểm cuối audio quay về thời điểm đầu audio.

Sau đó, tại mỗi thời điểm t , Bi-LSTM ghép hai trạng thái ẩn lại với nhau:

$$h_t = [h_{t_forward} ; h_{t_backward}]$$

Trong đó:

- $h_{t_forward}$ là trạng thái ẩn của LSTM hướng xuôi tại thời điểm t .
- $h_{t_backward}$ là trạng thái ẩn của LSTM hướng ngược tại thời điểm t .

Ký hiệu $[;]$ nghĩa là phép nối vector.

Nếu mỗi hướng LSTM có hidden size là 256, thì sau khi ghép hai hướng, vector đầu ra tại mỗi frame có kích thước:

$$256 + 256 = 512$$

Vì vậy, trong mô hình của đề tài, đầu ra của Bi-LSTM tại mỗi frame có kích thước 512 chiều.

Cách Bi-LSTM hoạt động

Giả sử đầu vào của mô hình là chuỗi spectrogram:

$$x_1, x_2, x_3, \dots, x_T$$

Bi-LSTM xử lý chuỗi này bằng hai LSTM riêng biệt.

- **LSTM hướng xuôi**

LSTM hướng xuôi đọc chuỗi theo thứ tự:

$$x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow \dots \rightarrow x_T$$

Ở mỗi thời điểm, LSTM hướng xuôi học ngữ cảnh từ quá khứ đến hiện tại. Điều này có nghĩa là trạng thái ẩn tại thời điểm t chứa thông tin từ các frame trước đó:

$$x_1, x_2, \dots, x_t$$

Trong bài toán Morse, hướng xuôi giúp mô hình biết trước đó đã xảy ra gì, ví dụ tone đã bắt đầu từ lúc nào, khoảng lặng trước đó dài bao lâu hoặc ký tự trước đó có thể đã kết thúc hay chưa.

- **LSTM hướng ngược**

LSTM hướng ngược đọc chuỗi theo thứ tự ngược lại:

$$x_T \rightarrow x_{\{T-1\}} \rightarrow x_{\{T-2\}} \rightarrow \dots \rightarrow x_1$$

Ở mỗi thời điểm, LSTM hướng ngược học ngữ cảnh từ tương lai quay về hiện tại. Điều này có nghĩa là trạng thái ẩn tại thời điểm t chứa thông tin từ các frame phía sau:

$$x_t, x_{\{t+1\}}, \dots, x_T$$

Trong bài toán Morse, hướng ngược giúp mô hình biết sau thời điểm hiện tại sẽ xảy ra gì. Ví dụ, nếu sau một đoạn tone là khoảng lặng dài, mô hình có thể hiểu rằng tone hiện tại có thể là kết thúc của một ký tự hoặc một từ.

- **Ghép hai hướng**

Sau khi có thông tin từ cả hai hướng, Bi-LSTM ghép hai trạng thái ẩn tại cùng thời điểm:

$$h_t = [h_{t_forward}; h_{t_backward}]$$

Nhờ đó, mỗi frame không chỉ được biểu diễn bằng thông tin quá khứ mà còn có cả thông tin tương lai. Điều này giúp mô hình hiểu ngữ cảnh đầy đủ hơn khi đưa ra dự đoán.

Vì sao Bi-LSTM phù hợp với bài toán Morse?

Bi-LSTM đặc biệt phù hợp với bài toán nhận dạng Morse vì tín hiệu Morse có tính phụ thuộc thời gian rất mạnh. Một frame riêng lẻ không đủ để xác định chính xác ký tự. Mô hình cần xem xét cả các frame trước và sau để hiểu được cấu trúc tín hiệu.

- **Phân biệt dot và dash chính xác hơn**

Dot và dash đều là tone ở cùng tần số, chỉ khác nhau ở thời lượng. Nếu mô hình chỉ nhìn một frame hiện tại, nó không thể biết tone này là dot hay dash. Bi-LSTM giúp mô hình quan sát toàn bộ ngữ cảnh xung quanh tone để xác định tone đó ngắn hay dài.

Ví dụ:

- Tone ngắn → có khả năng là dot.
- Tone dài → có khả năng là dash.
- Tone đang tiếp diễn → chưa nên kết luận quá sớm.

Nhờ có cả hướng xuôi và hướng ngược, mô hình có thể biết tone đã bắt đầu từ trước đó và sẽ kết thúc ở phía sau như thế nào.

• Nhận biết khoảng cách giữa các ký tự

Trong Morse, khoảng cách không chỉ là đoạn im lặng đơn giản. Độ dài của khoảng lặng mang ý nghĩa khác nhau:

- Khoảng lặng ngắn: khoảng cách giữa dot và dash trong cùng một ký tự.
- Khoảng lặng trung bình: khoảng cách giữa hai ký tự.
- Khoảng lặng dài: khoảng cách giữa hai từ.

Bi-LSTM giúp mô hình phân biệt các loại khoảng lặng này bằng cách quan sát cả trước và sau khoảng lặng. Điều này rất quan trọng để mô hình dự đoán đúng khoảng trắng trong văn bản đầu ra.

• Tận dụng ngữ cảnh hai chiều

Một ký tự Morse có thể được hiểu rõ hơn nếu biết các tín hiệu xung quanh nó. Ví dụ, khi mô hình đang xử lý một đoạn tone, thông tin phía trước cho biết tone này bắt đầu từ đâu, còn thông tin phía sau cho biết tone này kéo dài đến đâu. Kết hợp cả hai hướng giúp mô hình đưa ra biểu diễn chính xác hơn.

• Phù hợp với CTC Loss

CTC Loss không yêu cầu gán nhãn chính xác từng frame. Tuy nhiên, mô hình vẫn cần sinh ra chuỗi xác suất ký tự hợp lý theo thời gian. Bi-LSTM giúp đầu ra tại mỗi frame có nhiều ngữ cảnh hơn, từ đó hỗ trợ CTC học alignment tốt hơn giữa spectrogram và transcript.

Trong bài toán này, transcript chỉ là chuỗi văn bản cuối cùng, không có timestamp cho từng ký tự. Vì vậy, Bi-LSTM kết hợp với CTC là lựa chọn phù hợp, vì mô hình có thể tự học cách căn chỉnh ngầm giữa tín hiệu âm thanh và chuỗi ký tự.

So sánh LSTM một chiều và Bi-LSTM

LSTM một chiều chỉ đọc chuỗi từ đầu đến cuối. Do đó, tại thời điểm t , mô hình chỉ biết các thông tin từ quá khứ đến hiện tại. Điều này phù hợp với các hệ thống cần chạy thời gian thực, vì mô hình không cần biết tương lai.

Tuy nhiên, trong bài toán của đề tài, mục tiêu chính là nhận dạng file audio đã có sẵn, không bắt buộc phải xử lý real-time. Vì vậy, mô hình có thể xem toàn bộ đoạn audio trước khi dự đoán. Khi đó, Bi-LSTM có lợi thế hơn vì nó sử dụng được cả thông tin trước và sau mỗi frame.

So với LSTM một chiều, Bi-LSTM có các ưu điểm:

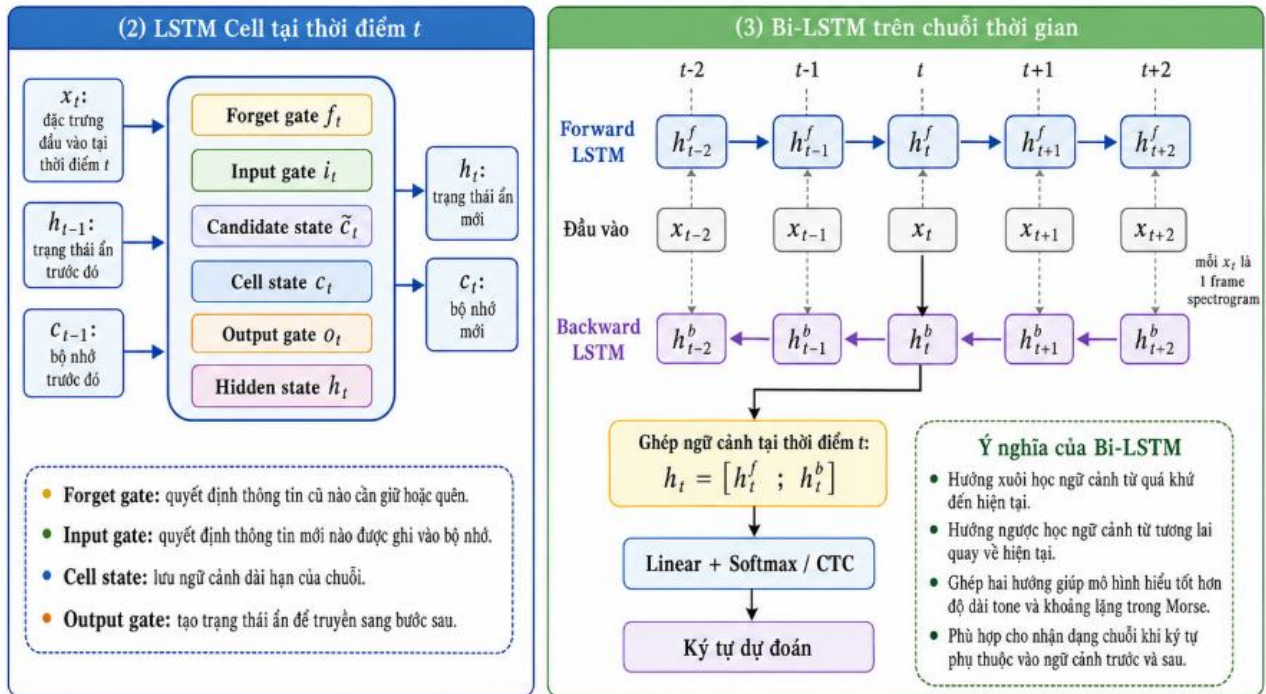
- Có nhiều ngữ cảnh hơn cho từng frame.
- Nhận biết ranh giới tone và khoảng lặng tốt hơn.
- Hỗ trợ CTC học alignment tốt hơn.

- Thường cho kết quả chính xác hơn trong bài toán nhận dạng chuỗi offline.

Nhược điểm của Bi-LSTM là mô hình nặng hơn và không phù hợp hoàn toàn với xử lý real-time, vì hướng ngược cần biết trước toàn bộ chuỗi. Tuy nhiên, với bài toán trong đề tài là xử lý file audio Morse, nhược điểm này không ảnh hưởng nhiều.

MINH HỌA CƠ CHẾ HOẠT ĐỘNG CỦA BI-LSTM

Bi-LSTM xử lý chuỗi theo cả hai hướng: từ trái sang phải và từ phải sang trái, sau đó ghép hai trạng thái ẩn để tăng ngữ cảnh cho từng thời điểm.



Cấu hình Bi-LSTM:

- Input size: 256
- Hidden size: 256
- Number of layers: 2
- Bidirectional: True
- Dropout: 0.3

Do Bi-LSTM có hai chiều, đầu ra tại mỗi frame có kích thước: $256 \times 2 = 512$

Vì vậy, sau Bi-LSTM, tensor có dạng: (batch_size, T', 512)

Trong đó T' là số frame thời gian sau khi đã đi qua Conv1D.

7.5. Lớp Linear để dự đoán xác suất ký tự

Sau Bi-LSTM, mỗi frame thời gian có vector đặc trưng 512 chiều. Lớp Linear cuối cùng ánh xạ vector này sang số lớp ký tự cần dự đoán.

Bộ ký tự của mô hình gồm:

- 26 chữ cái A-Z,
- 10 chữ số 0-9,
- 1 ký tự khoảng trắng,

- 1 blank token của CTC.
- Tổng số lớp là:
- $26 + 10 + 1 + 1 = 38$ lớp

Do đó, lớp Linear có dạng: $\text{Linear}(512 \rightarrow 38)$

Đầu ra của mô hình tại mỗi frame là phân phối xác suất trên 38 lớp. Sau đó, mô hình áp dụng `log_softmax` để tạo log-probability, phù hợp với yêu cầu đầu vào của `CTCLoss`.

Output cuối cùng của mô hình có dạng: $(\text{batch_size}, T, 38)$

Trong đó:

- `batch_size` là số mẫu trong một batch,
- `T` là số frame thời gian sau `Conv1D`,
- 38 là số lớp ký tự, bao gồm cả blank token.

7.6. Vì sao sử dụng CTC?

Trong bài toán nhận dạng Morse từ audio, ta chỉ có transcript cuối cùng của audio, ví dụ:

HELLO WORLD

Tuy nhiên, ta không biết chính xác:

- ký tự H bắt đầu ở frame nào,
- ký tự E kết thúc ở frame nào,
- khoảng trắng giữa hai từ nằm ở frame nào,
- mỗi dot hoặc dash tương ứng với đoạn thời gian nào.
- Nếu dùng cách huấn luyện thông thường, cần có alignment giữa từng frame audio và từng ký tự. Việc tạo alignment thủ công là rất khó và tốn thời gian.

CTC, viết tắt của **Connectionist Temporal Classification**, giải quyết vấn đề này. CTC cho phép huấn luyện mô hình sequence-to-sequence khi:

- đầu vào dài hơn đầu ra,
- không có thông tin căn chỉnh thời gian giữa input và output,
- chỉ có transcript cuối cùng.

CTC thêm một ký tự đặc biệt gọi là blank token. Trong quá trình học, mô hình có thể dự đoán nhiều frame là blank hoặc lặp lại một ký tự. Khi decode, CTC sẽ:

- gộp các ký tự lặp liên tiếp,
- loại bỏ blank token,
- trả về chuỗi ký tự cuối cùng.

Ví dụ, chuỗi frame dự đoán:

blank H H blank E blank L L blank L O sẽ được CTC decode thành: HELLO

Nhờ CTC, mô hình không cần biết chính xác mỗi ký tự nằm ở đâu trong audio. Đây là lý do CTC rất phù hợp với bài toán nhận dạng mã Morse.

7.7. Quá trình decode bằng CTC Greedy

Trong giai đoạn suy luận, mô hình sinh ra log-probability cho từng frame thời gian. Với mỗi frame, hệ thống chọn lớp có xác suất cao nhất. Đây gọi là greedy decoding.

Sau đó, chuỗi ký tự frame-level được xử lý theo quy tắc CTC:

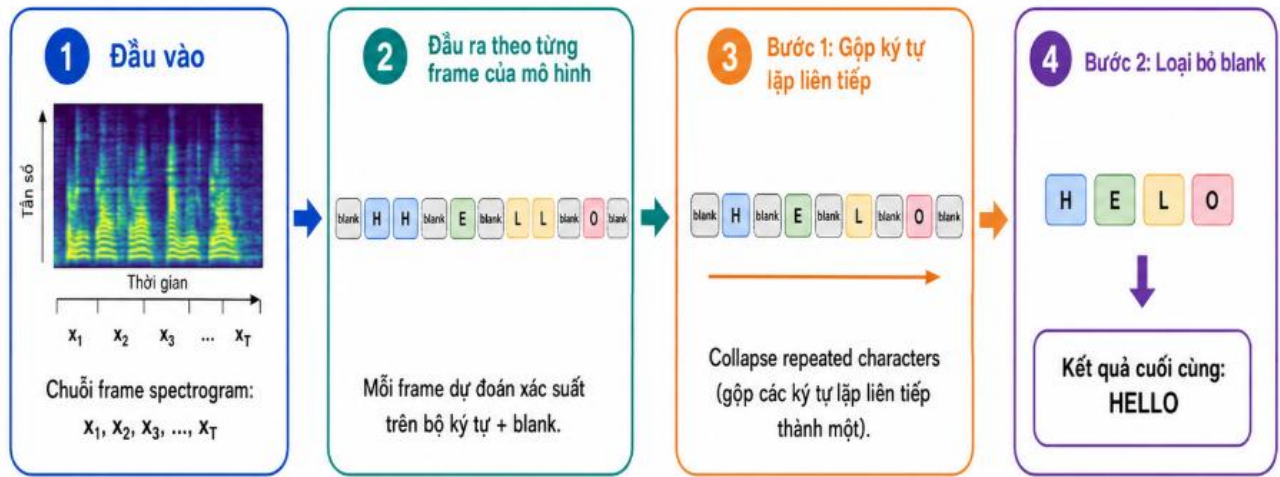
- loại bỏ các ký tự lặp liên tiếp,
- loại bỏ blank token,
- ghép các ký tự còn lại thành văn bản.

Ví dụ:

- Frame-level prediction:
blank T T blank H H E blank
- Sau khi gộp ký tự lặp:
blank T blank H E blank
- Sau khi bỏ blank: THE

Greedy decoding đơn giản, tốc độ nhanh và phù hợp với hệ thống demo. Trong các phiên bản phát triển tiếp theo, có thể thay greedy decoding bằng beam search để cải thiện độ chính xác, đặc biệt khi xử lý dữ liệu real hoặc audio nhiễu mạnh.

Minh họa cơ chế hoạt động của CTC



7.8. Kiến trúc tổng thể của mô hình

Kiến trúc tổng thể có thể mô tả theo các tầng sau:

- Input: Spectrogram có kích thước $(T, 21)$
- Conv1D Block 1: $21 \rightarrow 128$, kernel_size = 5, stride = 2, padding = 2
- BatchNorm1D
- ReLU
- Dropout
- Conv1D Block 2: $128 \rightarrow 256$, kernel_size = 5, stride = 1, padding = 2
- BatchNorm1D
- ReLU
- Dropout
- Bi-LSTM: input_size = 256, hidden_size = 256, num_layers = 2, bidirectional = True
- Linear: $512 \rightarrow 38$
- LogSoftmax
- CTC Loss trong huấn luyện hoặc CTC Greedy Decode khi suy luận

Chuỗi xử lý có thể viết gọn như sau:

$(T, 21) \rightarrow \text{Conv1D} \rightarrow (T', 256) \rightarrow \text{Bi-LSTM} \rightarrow (T', 512) \rightarrow \text{Linear} \rightarrow (T', 38) \rightarrow \text{CTC} \rightarrow \text{Text}$

8. Quy trình huấn luyện mô hình

Dữ liệu được chia stratified theo WPM để đảm bảo tập train và validation đều chứa đủ các tốc độ Morse. Mô hình được huấn luyện từ random initialization trên tập synthetic đã chuẩn bị sẵn. Trong mỗi epoch, hệ thống log cả loss và các metric chuỗi để quan sát chất lượng nhận dạng thực tế.

Tham số huấn luyện	Giá trị
Optimizer	AdamW
Learning rate	3e-4
Weight decay	0.01
Loss function	CTCLoss(blank=<blank>, zero_infinity=True)
Scheduler	CosineAnnealingLR
Batch size	16
Gradient accumulation	4 steps
Epoch tối đa	15
Early stopping	Patience = 5 theo validation CER
Train/Val split	90% / 10%, chia theo WPM

Quy trình huấn luyện gồm các bước chính sau:

Bước 1. Chuẩn bị dữ liệu

Bước 2. Chia tập train và validation

Bước 3. Forward pass

Trong mỗi batch, spectrogram được đưa qua mô hình:

- Conv1D trích xuất đặc trưng cục bộ,
- Bi-LSTM học quan hệ theo thời gian,
- Linear sinh log-probability trên từng ký tự,
- log_softmax tạo đầu ra phù hợp cho CTC.

Bước 4. Tính CTC Loss

CTC Loss so sánh chuỗi dự đoán frame-level của mô hình với transcript ground truth. Vì CTC tự xét nhiều khả năng alignment khác nhau giữa input và output, ta không cần timestamp cho từng ký tự.

Loss càng thấp nghĩa là mô hình càng dự đoán tốt chuỗi ký tự đúng.

Bước 5. Backpropagation và cập nhật trọng số

Sau khi tính loss, hệ thống thực hiện lan truyền ngược để tính gradient. Optimizer AdamW được dùng để cập nhật trọng số mô hình. AdamW phù hợp vì có cơ chế weight decay giúp hạn chế overfitting.

Ngoài ra, gradient clipping được sử dụng để tránh gradient quá lớn, đặc biệt khi train LSTM.

Bước 6. Đánh giá trên validation set

Sau mỗi epoch, mô hình được đánh giá trên validation set bằng các metric:

- Validation Loss,
- CER,
- WER,
- Exact Match,
- Pred/Ref Length Ratio,
- CER theo từng WPM.

Các metric này giúp đánh giá không chỉ mức độ giảm loss mà còn chất lượng văn bản đầu ra.

Bước 7. Lưu best model

Nếu validation CER của epoch hiện tại thấp hơn các epoch trước, model được lưu lại làm best model. Validation CER được chọn làm tiêu chí chính vì bài toán cần tối ưu chất lượng chuỗi ký tự dự đoán.

Best model sau đó được dùng cho bước inference và demo.

8.1. Quy trình một epoch

1. Nạp batch spectrogram và transcript đã mã hóa.
2. Padding batch theo chiều thời gian để các mẫu cùng kích thước.
3. Tính độ dài đầu vào sau Conv1D để đưa vào CTC Loss.
4. Forward qua mô hình để thu log-probabilities.
5. Tính CTC Loss giữa output và transcript.
6. Backward với gradient accumulation để giảm áp lực bộ nhớ GPU.
7. Cập nhật trọng số bằng AdamW.
8. Đánh giá trên validation set bằng Loss, CER, WER, Exact Match và CER theo WPM.
9. Lưu best model nếu validation CER giảm.

9. Hệ thống đánh giá và metrics

Để đánh giá đầy đủ, đề tài không chỉ dùng loss mà còn sử dụng các metric trực tiếp trên chuỗi văn bản. Điều này quan trọng vì CTC Loss giảm không phải lúc nào cũng phản ánh đầy đủ chất lượng transcript cuối cùng.

- CTC Loss

CTC Loss là hàm mất mát chính để huấn luyện mô hình. Loss này đo mức độ sai khác giữa đầu ra frame-level của mô hình và transcript đích. Loss giảm dần qua các epoch cho thấy mô hình đang học tốt hơn.

- CER

CER, hay Character Error Rate, là metric chính trong đề tài. CER được tính bằng khoảng cách Levenshtein giữa chuỗi dự đoán và chuỗi gốc, chia cho độ dài chuỗi gốc.

$$\text{CER} = \text{edit_distance}(\text{PRED}, \text{REF}) / \text{length}(\text{REF})$$

CER càng thấp thì mô hình nhận dạng ký tự càng chính xác.

- **WER**

WER, hay Word Error Rate, đo lỗi ở cấp độ từ. WER đặc biệt hữu ích khi đánh giá câu hoàn chỉnh, vì một lỗi ký tự có thể làm sai cả từ.

$$\text{WER} = \text{edit_distance}(\text{PRED_words}, \text{REF_words}) / \text{number_of_REF_words}$$

- **Exact Match**

Exact Match đo tỷ lệ mẫu mà toàn bộ câu được dự đoán đúng hoàn toàn. Đây là metric rất nghiêm ngặt. Chỉ cần sai một ký tự, mẫu đó không được tính là đúng hoàn toàn.

- **Pred/Ref Length Ratio**

Metric này so sánh độ dài chuỗi dự đoán với độ dài chuỗi gốc. Nếu tỷ lệ quá thấp, mô hình có xu hướng dự đoán thiếu ký tự. Nếu tỷ lệ quá cao, mô hình có xu hướng sinh thừa ký tự.

- **CER theo WPM**

Dữ liệu Morse có nhiều tốc độ khác nhau. Vì vậy, ngoài CER tổng thể, hệ thống còn tính CER theo từng nhóm WPM. Metric này giúp xác định mô hình đang hoạt động tốt hoặc kém ở tốc độ nào.

Metric	Công thức / Ý nghĩa	Vai trò
Train CTC Loss	Loss trung bình trên tập train	Theo dõi quá trình tối ưu
Validation CTC Loss	Loss trung bình trên tập validation	Theo dõi khả năng tổng quát
CER	Edit distance ký tự / độ dài ref	Metric chính cho nhận dạng Morse
WER	Edit distance theo từ / số từ ref	Đánh giá lỗi ở cấp từ
Exact Match	Tỷ lệ câu predict giống hoàn toàn ref	Đánh giá nghiêm ngặt chất lượng câu
Pred/Ref Length Ratio	Độ dài pred / độ dài ref	Phát hiện mô hình predict quá ngắn hoặc quá dài
CER theo WPM	CER tách riêng từng tốc độ	Tìm tốc độ Morse còn yếu

9.1. Biểu đồ trực quan

- Biểu đồ Train Loss và Validation Loss theo epoch.
- Biểu đồ Validation CER và WER theo epoch.
- Biểu đồ Exact Match theo epoch.
- Biểu đồ CER cuối cùng theo từng WPM.
- Bảng sample REF/PRED sau mỗi epoch để kiểm tra lỗi định tính.
- Hình spectrogram demo đầu vào mô hình khi chạy inference.

10. Quy trình inference và lưu kết quả

Sau khi train xong, checkpoint tốt nhất được dùng để dự đoán trên file audio mới. Quy trình inference gồm: đọc audio, trích xuất spectrogram, forward qua mô hình, CTC greedy decode và lưu kết quả vào GCS.

```
Audio WAV → Spectrogram (T,21) → Conv1D + Bi-LSTM → CTC Greedy Decode → Predicted text
```

File / thư mục	Nội dung
test/audio/<tên audio>/<tên audio>.wav	File audio đầu vào hoặc audio test được sinh
test/audio/<tên audio>/spectrogram.npy	Đặc trưng spectrogram đưa vào mô hình
test/audio/<tên audio>/metadata.json	Metadata khi sinh audio synthetic test
test/audio/<tên audio>/result.json	Kết quả dự đoán, REF, PRED, CER, WER nếu có REF

11. Kết quả thực nghiệm

Trên dữ liệu synthetic_realistic_v1, mô hình học được quy luật tín hiệu Morse và có thể dự đoán trực tiếp transcript từ audio. Trong lần thực nghiệm với tập synthetic 50.000 mẫu, checkpoint tốt nhất đã đạt validation CER rất thấp trên dữ liệu synthetic. Kết quả này cho thấy pipeline sinh dữ liệu, trích xuất đặc trưng, huấn luyện CTC và suy luận hoạt động đúng mục tiêu đề tài.

Chỉ số	Kết quả ghi nhận
Số mẫu synthetic	50.000
Validation split	Khoảng 10% dữ liệu, chia theo WPM
CER tốt nhất trên validation synthetic	Khoảng 0,0088, tương đương 0,88% trong lần chạy đã lưu checkpoint
Mô hình lưu tốt nhất	models/synthetic_only/model_synthetic_ctc_best.pt
Output demo	test/audio/<tên audio>/result.json

Kết quả training trên epoch tốt nhất:

```

=====
... EPOCH 14/15
=====
Train Loss: 0.0115 | Val Loss: 0.0131 | CER: 0.0044 | WER: 0.0444 | Exact: 0.8989 | Pred/Ref Len: 0.998 | Time: 432.5s

CER by WPM:
WPM 10 | CER=0.0053 | WER=0.0530 | Exact=0.9426 | N=122
WPM 13 | CER=0.0076 | WER=0.0759 | Exact=0.9113 | N=203
WPM 15 | CER=0.0063 | WER=0.0637 | Exact=0.9192 | N=297
WPM 18 | CER=0.0044 | WER=0.0429 | Exact=0.9258 | N=647
WPM 20 | CER=0.0026 | WER=0.0269 | Exact=0.9473 | N=664
WPM 25 | CER=0.0051 | WER=0.0497 | Exact=0.8864 | N=810
WPM 30 | CER=0.0041 | WER=0.0415 | Exact=0.8954 | N=813
WPM 35 | CER=0.0044 | WER=0.0451 | Exact=0.8723 | N=838
WPM 40 | CER=0.0047 | WER=0.0462 | Exact=0.8522 | N=609

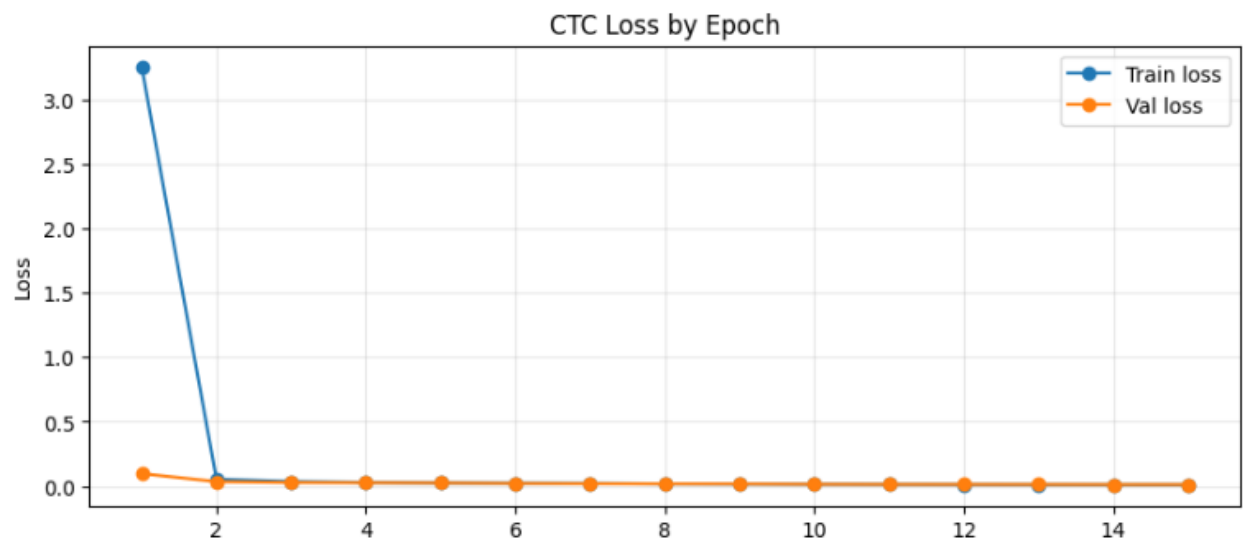
Sample REF/PRED:
-----
FILE: synreal_v1_000806_30wpm.wav | WPM=30 | CER=0.0000 | WER=0.0000
REF : 1 INCH TO 3 INCHES ACROSS
PRED: 1 INCH TO 3 INCHES ACROSS
-----
FILE: synreal_v1_030914_25wpm.wav | WPM=25 | CER=0.0323 | WER=0.3333
REF : BLACK PAN HEAD SCREWS WHICH ARE
PRED: BLACK PAN HEAD SCREWS WHICHARE
-----
FILE: synreal_v1_047957_35wpm.wav | WPM=35 | CER=0.0000 | WER=0.0000
REF : ON THE LCD PANEL THERE ARE A FEW
PRED: ON THE LCD PANEL THERE ARE A FEW
-----
FILE: synreal_v1_007983_25wpm.wav | WPM=25 | CER=0.0000 | WER=0.0000
REF : WERE ON HAND C201 AND
PRED: WERE ON HAND C201 AND

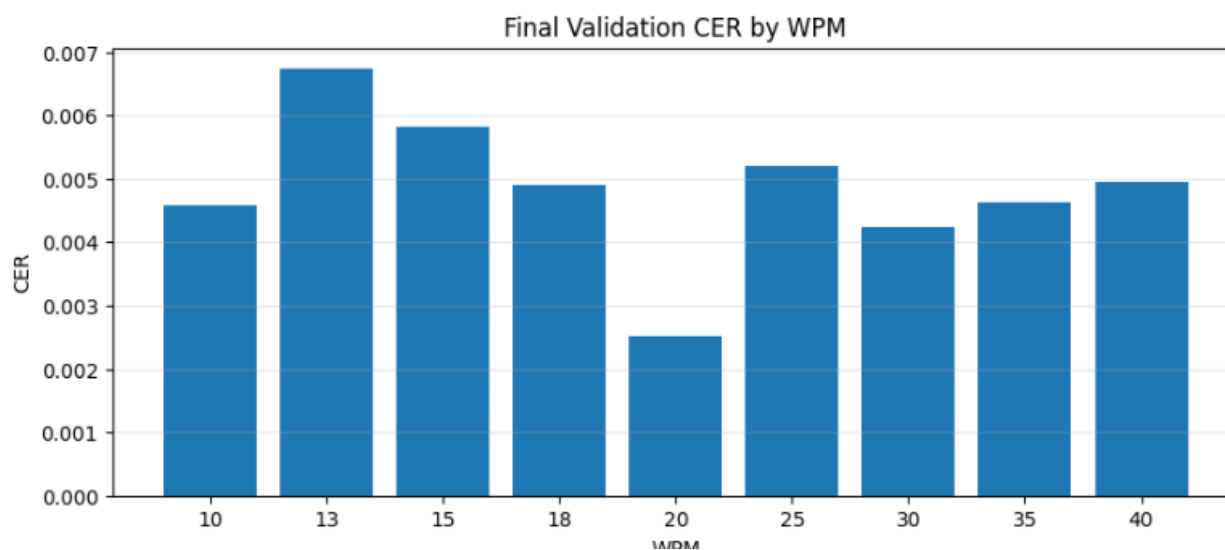
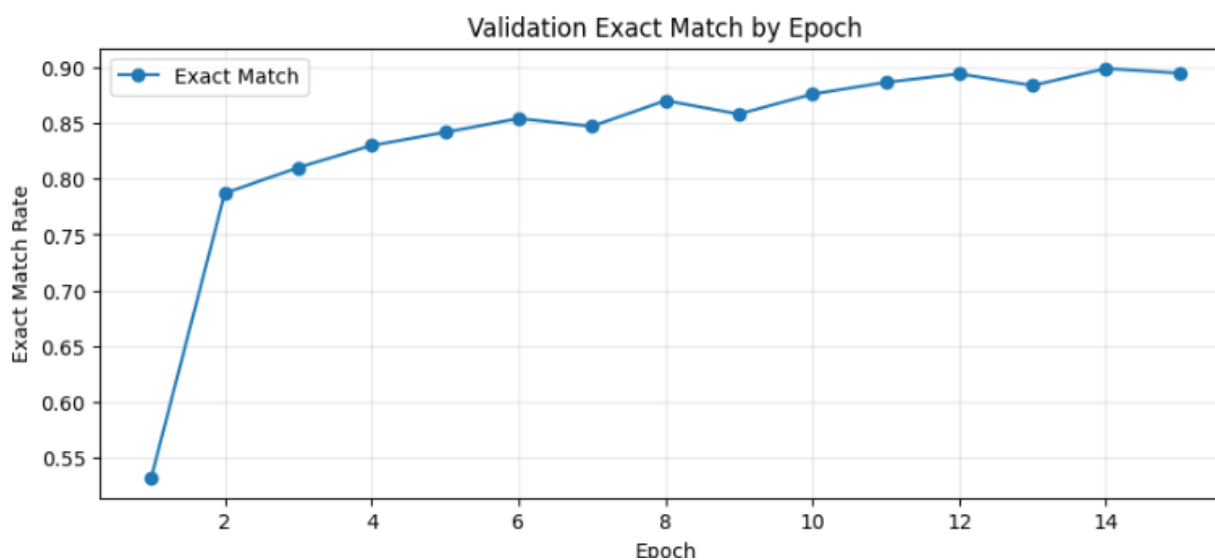
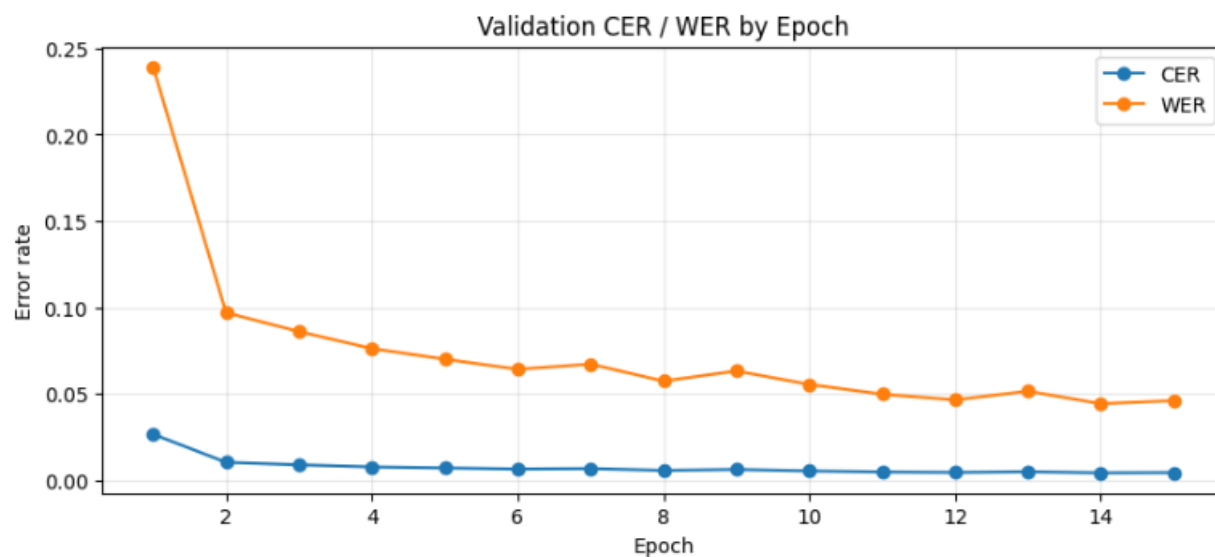
✓ New best model saved | CER=0.0044 | epoch=14
GCS: gs://dl-ptit/models/synthetic_only/model_synthetic_ctc_best.pt

```

Kết quả training trên toàn bộ epoch:

	epoch	train_loss	val_loss	cer	wer	exact_match	pred_ref_len_ratio
0	1	3.252130	0.097777	0.026766	0.239176	0.532481	0.974921
1	2	0.054711	0.035682	0.010530	0.097073	0.787128	0.993672
2	3	0.036614	0.028909	0.009071	0.086160	0.810114	0.995241
3	4	0.029660	0.027057	0.007861	0.076311	0.829902	0.998541
4	5	0.025753	0.025487	0.007289	0.070204	0.841695	0.997551
5	6	0.023203	0.022276	0.006636	0.064334	0.854088	0.998922
6	7	0.021724	0.019987	0.006900	0.067329	0.847092	0.995072
7	8	0.018593	0.018481	0.005844	0.057519	0.870078	0.997800
8	9	0.016979	0.017910	0.006387	0.063349	0.858085	0.994896
9	10	0.015612	0.015259	0.005551	0.055588	0.875874	0.995959
10	11	0.014269	0.014335	0.004950	0.049915	0.886468	0.997059
11	12	0.013052	0.014080	0.004671	0.046645	0.894064	0.998379
12	13	0.012133	0.013793	0.005089	0.051649	0.883670	0.996539
13	14	0.011476	0.013051	0.004422	0.044400	0.898861	0.997800
14	15	0.011057	0.013021	0.004576	0.046173	0.894663	0.997338





Kết quả thực nghiệm trên 1 file audio bất kì (được sinh ra):

```
=====
DEMO: GENERATE AUDIO → SPEC → PREDICT → COMPARE
=====
Demo audio: {'wav_path': '/content/morse_synthetic_project/demo/demo_morse.wav', 'ref': 'THE QUICK BROWN FOX JUMPS OVER'}
=====
REF : THE QUICK BROWN FOX JUMPS OVER 123
PRED: THE QUICK BROWN FOX JUMPS OVER 123
-----
REF_LEN : 34
PRED_LEN: 34
CER      : 0.0000
WER      : 0.0000
EXACT    : 1
=====
```

12. Cấu trúc thư mục và tổ chức trên GCS

```
gs://dl-ptit/
├── data/
│   ├── synthetic_realistic_v1/
│   │   ├── audio/
│   │   ├── specs/
│   │   └── annotations/
│   │       ├── synthetic_realistic_v1_labels.json
│   │       ├── synthetic_realistic_v1_stats.json
│   │       └── synthetic_realistic_v1_corpus.json
│   └── models/
│       └── synthetic_only/
│           ├── model_synthetic_ctc_best.pt
│           ├── model_synthetic_ctc_last.pt
│           └── train_history_synthetic_ctc.json
└── test/
    ├── audio/
    │   └── <tên audio>/
    │       ├── <tên audio>.wav
    │       ├── metadata.json
    │       ├── spectrogram.npy
    │       └── result.json
```

13. Hướng phát triển với dữ liệu real

Mặc dù phiên bản hiện tại chỉ dùng dữ liệu synthetic, hướng phát triển tiếp theo là mở rộng sang dữ liệu Morse thực tế. Đây là bước khó hơn vì dữ liệu real thường có nhiễu radio, độ lệch tốc độ, khoảng nghỉ không đều và transcript có thể không khớp tuyệt đối với đoạn audio được cắt.

Hướng phát triển	Mô tả
Thu thập real audio	Sử dụng nguồn audio Morse thực tế như W1AW hoặc bản thu radio.
Chuẩn hóa real audio	Resample 16 kHz, mono, detect tone frequency, chuẩn hóa biên độ.
Tạo real chunks sạch	Cắt audio thành đoạn ngắn 6-15 giây và đảm bảo transcript khớp với audio.
Lọc nhiễu	Dùng model hiện tại để decode, so sánh với transcript và chỉ giữ mẫu có độ tin cậy cao.

Fine-tune nhẹ	Dùng tập real nhỏ nhưng sạch để fine-tune với learning rate thấp.
Đánh giá riêng real	Báo cáo CER/WER trên real theo từng WPM và mức nhiễu.

13.1. Khó khăn khi mở rộng sang real data

- Audio real thường dài, có thể kéo dài hàng chục phút, gây khó khăn khi đưa trực tiếp vào mô hình.
- Cần cắt thành chunk ngắn nhưng vẫn phải giữ transcript chính xác cho từng chunk.
- Tốc độ WPM thực tế và khoảng nghỉ có thể khác lý thuyết.
- Nhiều radio, fading, QRM/QRN và chất lượng thu âm làm phổ âm thanh khác synthetic.
- Nếu label chunk sai, fine-tune supervised có thể làm mô hình học sai.
- Cần bước kiểm tra thủ công hoặc model-assisted filtering để tạo tập real đủ sạch.

14. Khó khăn, rủi ro và giải pháp

Rủi ro	Ảnh hưởng	Giải pháp
Mô hình overfit synthetic	Predict tốt trên synthetic nhưng kém trên real	Tăng augmentation và bổ sung real clean ở giai đoạn sau
Dữ liệu synthetic mất cân bằng WPM	Một số tốc độ có CER cao hơn	Theo dõi CER theo WPM và sinh bổ sung nhóm yếu
Audio ngoài thực tế không ở 800 Hz	Spectrogram 21 bins quanh 800 Hz có thể bỏ lỡ tín hiệu	Thêm bước detect pitch và frequency normalization
Greedy CTC decode chưa tối ưu	Một số lỗi khoảng trắng hoặc ký tự gần nhau	Thử beam search hoặc language model đơn giản
Colab hết phiên hoặc mất GPU	Huấn luyện bị gián đoạn	Lưu last model, best model và history lên GCS sau mỗi epoch
File audio test khác phân phối train	Prediction kém	Đưa metadata cảnh báo và mở rộng augmentation khi cần

15. Kết luận

Đề tài đã xây dựng một pipeline hoàn chỉnh cho nhận dạng mã Morse từ âm thanh trên dữ liệu tổng hợp. Pipeline bao gồm sinh dữ liệu, chuẩn hóa audio, trích xuất narrow spectrogram, huấn luyện mô hình Conv1D + Bi-LSTM + CTC, đánh giá bằng các metric chuỗi và suy luận trên file audio mới.

Lựa chọn spectrogram 21 bins quanh 800 Hz giúp mô hình tập trung vào đặc trưng quan trọng của tín hiệu Morse, giảm số chiều đầu vào và tăng tốc huấn luyện. CTC Loss cho phép mô hình học alignment giữa âm thanh và transcript mà không cần nhãn thời gian chi tiết. Với tập synthetic 50.000 mẫu, mô hình đạt hiệu quả cao trên validation synthetic, đủ đáp ứng phạm vi hiện tại của đề tài.

Trong tương lai, hệ thống có thể mở rộng sang dữ liệu real bằng cách xây dựng tập real clean nhỏ, kiểm soát chặt chất lượng nhãn, sau đó fine-tune nhẹ và đánh giá riêng trên các điều kiện thu âm thực tế.

Pred/Ref Length Ratio	$\text{len(PRED)} / \text{len(REF)}$
-----------------------	--------------------------------------